# Plural

## Identity and Profile Management Service

*One Identity, Many Dimensions*


1. Introduction....2
   1.1 Selected Template....2
   1.2 Challenge Overview: Digital Identity Fragmentation....2
   1.3 Solution Overview....3
   1.4 Solution Scope....3
   1.5 Motivation....4
2. Literature Review....4
   2.1 Introduction....4
   2.2 Defining Digital Identity....4
   2.3 Comparative Analysis of Existing Systems....5
3. Design & Plan....8
   3.1 Project Overview....8
   3.2 Domain and Target Audience....8
   3.3 System Architecture....9
   3.4 Design and Architectural Choices....9
   3.5 Initial Database Design....11
   3.6 Work Plan....12
   3.7 Success Criteria & Evaluation Methodology....13
4. Implementation....14
   4.1 Current Technical Stack....14
   4.2 REST API....14
   4.3 Privacy Enforcement....15
   4.4 Web Interface....15
   4.5 Admin Panel....16
   4.6 Flexible Data Model....16
5. Evaluation....16
   5.1 Testing Approach....16
   5.2 What the Tests Cover....16
   5.3 Results Against Success Criteria....17
   5.4 Limitations and Improvement Plan....18
6. Conclusion....19
   6.1 Summary....19
   6.2 What I Would Do Next....19
   6.3 Reflection....19
References....20

# *1. Introduction*

## 1.1 Selected Template

7.1 Project Idea 1: Identity and profile management API

## 1.2 Challenge Overview: Digital Identity Fragmentation

Modern digital platforms are designed to capture or focus on a certain dimensions of human identities. When users engage with these platforms, they are expected to show the identity, the dimension and the role suited to that environment. For example, LinkedIn is designed to highlight the professional persona; it is a space where sharing an Azure certification is appropriate, but sharing a gaming high score would be out of context. Conversely, a platform like Steam captures the recreational “gamer” dimension, while Goodreads captures the intellectual “reader” dimension.

However, while these dimensions are distinct in function, focus and interest they belong to a single human entity. The current digital infrastructure fails to reflect this unity. These platforms operate as isolated silos, unaware of the user’s existence outside their specific boundary. This fragmentation is not just a technical inconvenience. Research shows that people naturally present different sides of themselves in different social contexts [1], and that current platforms force all these audiences into one view, a phenomenon known as “context collapse” [2].

**Centralized Identity Management:** The user lacks a centralized mechanism to observe and manage these fragmented selves, Instead, they are forced to manually duplicate data and manage different profiles across different systems, leading to administrative friction and a loss of holistic control over their own scattered digital footprint.

**Public multidimensional Identity:** This presents another challenge, particularly for individuals with multifaceted talents and diverse interests. These users want to reflect a fair, multidimensional public picture of themselves; for example, they want to say: ‘I am a professional software engineer, but I am also a reader interested in history and philosophy, and a gamer who loves Chess and League of Legends’; A definition that spans different dimensions.

**Private Identity:** Equally important is the need for privacy and boundary enforcement. While users often want to show multiple sides of themselves, they also need to keep other sides strictly confidential. For instance, a user must share their legal identity for government services but may need to hide that same identity from online gaming communities for safety. Currently, users lack the granular control to ensure that sensitive private attributes (like a legal name) do not leak into inappropriate contexts.

**Conclusion:** To bridge these gaps, we need a centralized system where users can manage their identity dimensions, choosing what to share publicly and what to keep private, while integrating seamlessly with existing external platforms via a RESTful API.

## 1.3 Solution Overview

The project aims for implementing a centralized place for orchestrating the digital multidimensional identity, by allowing the users to manage the identity as an aggregation of distinct personas, each persona represents a particular dimension (e.g. Legal, Reader, Gamer, Worker), and has its own set of attributes and relevant in a certain context, each persona is set to be public or private (protected access)

Each user will have a general profile that presents all of the user's public personas and what they want to share about themselves (e.g., professional persona, reader persona, gamer persona), while strictly hiding confidential personas (e.g., legal persona).

The system responds dynamically based on the context. If a recruiter queries the profile, they will see only LinkedIn and GitHub accounts; however, if a gamer friend explores the platform, they may see the gamer identity. This means that a single user endpoint can yield completely different JSON responses depending on the negotiation context.

**The Final Outcome:** The system provides both a REST API for programmatic access and a web interface (GUI) that allows users to manage their personas through a browser.

## 1.4 Solution Scope

It's important to define a clear scope to avoid any scope creep

**What this project IS:** The application focuses on the Representation Layer (how the user wants to present themselves in different contexts). For the current iteration, the system includes a simple built-in session-based authentication to get the core persona management working. In future iterations, I plan to delegate authentication to an external provider like AWS Cognito to enhance integration with other systems and improve security.

**What this project is NOT:** To clarify the specific contribution of this work, it is important to distinguish it from existing terms:

- **This is not an IAM Service:** While the terminology (i.e. identity management) is similar to cloud services like AWS IAM or Azure AD, they serve different purposes. Cloud IAM manages permissions for infrastructure resources, whereas this project manages user profile contexts, they focus on authentication while this project focuses on representation.
- **This is not a Single Sign-On (SSO) Solution:** The goal is not to provide a mechanism where users sign in once to access multiple platforms without sharing credentials.

While the application could integrate with external systems like IAM and SSO services, it is designed from the ground up to serve a single purpose: managing and projecting the different representations and dimensions (personas) of the user.

## 1.5 Motivation

**Personal Motivation:** I am deeply interested in backend web development, data, security and identity management systems, I enjoy building APIs that could be used and integrated in different systems, I enjoy system integration. My passion lies in the challenge of system integration: building secure, scalable APIs that act as bridges between disconnected platforms. For me, this project is not just a requirement; it is an opportunity to build the kind of structural, data-driven tool that I personally enjoy using.

**Domain Opportunity:** The current software market is already saturated with strong solutions for Authentication ("Who are you?") and Authorization ("What are you allowed to do?") [3][4][5]. However, there is a significant gap when it comes to Representation ("How do you appear in this context?").

---

# *2. Literature Review*

## 2.1 Introduction

Digital identity representation exists at the intersection of human sociological behavior and software engineering architecture. To propose a robust solution, it is necessary to examine research from both domains: the human studies that define why we fragment our identities, and the technical studies that define how current systems manage them. This chapter evaluates the current digital status quo, analyzing relevant projects, protocols, and services to identify the specific architectural gap this project intends to fill.

## 2.2 Defining Digital Identity

Before designing a system to manage identity, we must establish a working definition of the term.

From a technical perspective, industry leaders offer converging definitions. Oracle defines digital identity as "a collection of data points that comprise the characteristics, attributes, and activities that identify an entity" [6]. Cloudflare views it as "the recorded set of measurable characteristics by which a computer can identify an external entity" [7]. IBM defines it as "a profile or set of information tied to a specific user, machine or other entity in an IT ecosystem" [8].

These definitions treat identity as a single fixed profile. But that does not match how people actually behave. Goffman (1959) argued that people naturally perform different roles in different social situations, we act differently in a job interview than we do with friends, not because we are being dishonest, but because different settings call for different presentations of ourselves [1]. This idea that identity is not one thing but many context-dependent facets is central to this project.

Clauß and Kohntopp (2001) brought this idea into computer science with the concept of "partial

identities", the different subsets of a person's attributes used in different contexts [9]. They argued that identity management systems should let users control which partial identity is shown to which party. This is essentially what this project calls a "Persona."

The problem is that most current platforms do the opposite. Marwick and boyd (2011) describe "context collapse", when social media flattens all your audiences into one, so a post meant for friends is also visible to your employer [2]. The user loses the ability to show different sides of themselves to different people.

Nissenbaum (2004) explains why this matters from a privacy perspective. Her theory of "contextual integrity" says that privacy is not about hiding information, but about information flowing to the right context [10]. Sharing your legal name with a bank is normal; having it exposed in a gaming forum is a violation of context. This framework supports the design decision behind Plural's per-persona privacy controls, each persona belongs to a context, and the system enforces boundaries between them.

**Synthesis:** From these definitions, we can conclude that Digital Identity is fundamentally a set of information and attributes representing a profile. But the academic literature shows that this set is not one thing, it is made up of context-dependent subsets (what Clauß and Kohntopp call "partial identities" [9] and what this project calls "Personas"). Current platforms collapse these contexts [2], and the goal of this project is to restore those boundaries by giving users control over which part of their identity is visible in which context.

## 2.3 Comparative Analysis of Existing Systems

The current technological landscape is full by numerous systems, technologies and services that address distinct but adjacent challenges. However they solve slightly different set of problems and they solve them from different angles or using fundamentally different approaches, so it's a good idea to take a look at some of them.

### *2.3.1 Cloud-based Identity and Access Management Services (IAM)*

The dominant standard in the current web ecosystem is Identity and Access Management (IAM). IAM systems manage digital identities and control user access to resources, ensuring that the right people can access the right systems at the right time [3][4][5]. In practice, this means IAM is built around two core questions: Authentication ("Is this user who they claim to be?") and Authorization ("Is this user allowed to access this resource?").

Neither of these addresses a third question that this project focuses on: Representation ("Which dimension of this user should appear in this context?"). An IAM system can verify that a user is "John Doe" and that he is allowed to access a resource, but it cannot present him as a "gamer" to one visitor and as a "lawyer" to another. The data model of IAM systems does not support multiple contextual projections of the same identity.

It is worth noting that Cameron (2005) proposed seven "Laws of Identity" for building identity systems, including principles like user control, minimal disclosure, and support for multiple

operators [11]. These principles recognise that users should only reveal the minimum information necessary for each interaction, an idea that aligns with this project's approach of exposing only relevant personas per context. However, mainstream IAM implementations have focused on the authentication and authorisation aspects rather than on contextual self-presentation.

**Integration not Replacement:** The proposed project is not a replacement for IAM but a complement to it. Architecturally, the system functions as a representation layer that sits on top of existing IAM infrastructure. IAM handles the security question (who are you?), while Plural handles the presentation question (which version of you should this visitor see?). The two layers are designed to work together, not compete.

### *2.3.2 Blockchain-Based Identity Management System (IDMS)*

Several researches focuses on Decentralized Identity Management Systems (IDMS), often leveraging blockchain technology to address privacy and ownership concerns.

Alanzi and Alkhatib (2025) discuss the privacy and security threats related to traditional systems: "Traditional IDMS relies on a third party to store user information and authenticate the user. However, this approach poses threats to user privacy and increases the risk of single point of failure (SPOF), user tracking, and data unavailability" [12].

They suggest that modern blockchain-based systems could overcome these challenges: "In contrast, decentralized IDMSs that use blockchain technology offer potential solutions to these issues as they offer powerful features including immutability, transparency, anonymity, and decentralization" [12].

However, the authors are realistic about the limitations of this technology: "Despite its advantages, blockchain technology also suffers from limitations related to performance, third-party control, weak authentication, and data leakages. Furthermore, some blockchain-based IDMSs still exhibit centralization issues, which can compromise user privacy and create SPOF risks" [12].

**Blockchain Limitation:** Despite these advantages, blockchain solutions suffer from low adoption rates, performance bottlenecks, weak authentication mechanisms (private key management), and complexity. Furthermore, they often require a complete paradigm shift, both the user and the visiting platform must adopt Web3 standards (Wallets, Smart Contracts).

**Differentiation:** Both blockchain-based IDMS and this project share the goal of giving users more control over their data. However, they differ fundamentally in approach. Blockchain systems require both the user and every consuming platform to adopt Web3 infrastructure (wallets, smart contracts, decentralised protocols). This creates a high adoption barrier. The proposed project takes a pragmatic approach: it uses standard RESTful APIs that any existing web application can consume without adopting new protocols. A game client or a recruitment platform can query a user's persona via a simple HTTP request, no blockchain integration required. The trade-off is that the system is centralised (the user trusts the Plural server), but in return it gains simplicity, performance, and compatibility with the existing web.

### *2.3.3 Federated Identity and Social Login (OAuth 2.0 & OIDC)*

Federated Identity handles cross-platform identity through protocols like OAuth 2.0 and OpenID Connect (OIDC), popularly known as “Log in with Google” or “Log in with Facebook.” The core idea is that a user authenticates once with a trusted Identity Provider (IdP) and can then access multiple third-party applications without creating separate accounts for each one [13][14][15].

In practice, when a user logs in via an IdP, the application receives a “User Info” bundle, typically a name, email, and profile picture. The user gets convenience (one login for many services), and the application gets verified identity data without handling passwords itself.

**Differentiation:** Federated access and this project address related but distinct problems. Federated access solves the problem of using *the same identity* across different platforms, one login for many services. This project solves the opposite problem: presenting *different facets of the same identity* depending on the context. OAuth can tell a service “this is John Doe,” but it cannot tell one service “show John’s gaming profile” and another service “show John’s professional profile.” The User Info bundle shared by an IdP is a fixed set of attributes (name, email, picture); it has no concept of contextual personas or privacy toggles. Plural fills this gap by adding a representation layer on top of the authentication layer.

### *2.3.4 Identity Aggregators (Link-in-Bio Tools)*

While the previous sections covered infrastructure-level solutions, there is also a user-facing category of tools that addresses identity fragmentation directly. Platforms such as Linktree, Carrd, and About.me, commonly known as “link-in-bio” tools, allow users to create a single landing page containing links to all their profiles across different platforms (e.g., Instagram, LinkedIn, GitHub, Spotify). They serve as a “digital business card” that routes visitors to the user’s presence elsewhere on the web.

**Limitation:** Despite their popularity, these tools have significant limitations regarding the goals of this project:

- **Static Landing Pages:** These pages are static; they present the exact same list of links to every visitor. The user cannot choose to show their LinkedIn to a recruiter while hiding their Spotify playlist.
- **Lack of Granular Control:** The user lacks the ability to define distinct “Personas.” There is no mechanism to group attributes into logical dimensions (e.g., a “Work Mode” vs. a “Social Mode”)
- **Technical Isolation:** These tools are designed for human navigation, not system integration. They lack a RESTful API to support programmatic integration with existing systems, making them unsuitable for the architectural role of an identity middleware.

**Differentiation:** Identity Aggregators and this project both attempt to centralise a user’s fragmented digital presence, but they differ in three key ways. First, aggregators are static, every

visitor sees the same page, whereas Plural returns different data depending on the access context. Second, aggregators lack privacy controls, there is no mechanism to hide certain links from certain visitors, whereas Plural enforces public/private boundaries per persona. Third, aggregators are designed for human browsing (HTML pages), not machine consumption; they lack APIs that would allow a third-party application to programmatically retrieve a user's gamer profile or professional credentials. Plural addresses all three of these limitations.

---

## *3. Design & Plan*

### 3.1 Project Overview

This project is a web application called **Plural** that allows users to manage multiple digital personas from a single account. The core concept is simple: a user signs up once, and then creates as many personas as they need, each representing a different dimension of their identity (e.g., "Professional," "Gamer," "Legal"). Each persona has its own set of attributes (stored as flexible JSON data) and a visibility setting: **public** (visible to anyone) or **private** (accessible only with a secret access token).

The system has two interfaces:

1. **A REST API**, the primary product. Third-party applications, recruiters, or other systems can query a user's profile programmatically. When they do, the API returns only the user's public personas. Private personas are completely hidden unless the caller provides the correct access token in the request header. This means the same API endpoint can return different data depending on who is asking.
2. **A web interface**, a browser-based dashboard where users can log in, create/edit/delete personas, toggle visibility, and share their public profile via a URL (`/u/{username}`).

The architecture follows a standard three-tier pattern: the FastAPI application serves both the API and the web interface, backed by a PostgreSQL database. The privacy enforcement logic sits in the application layer, filtering persona data before it reaches the response.

### 3.2 Domain and Target Audience

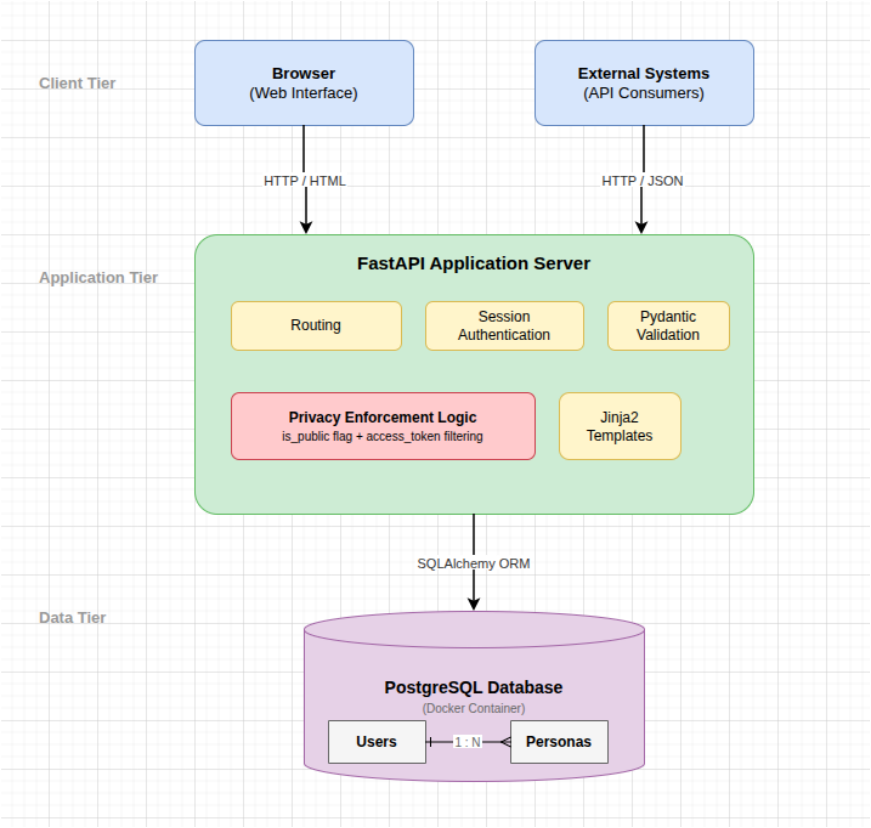
The project operates within the domain of Privacy Engineering and Identity Management (IdM). It addresses the friction between modern privacy needs and the rigid "Single Sign-On" structures of the current web. The academic literature supports the need for this kind of system: Clauß and Kohntopp (2001) argued that identity management systems should support "partial identities" that users can selectively disclose [9], and Nissenbaum (2004) showed that privacy violations occur when information flows outside its intended context [10]. This project is a practical attempt to implement those principles.

**Target Audience:**

- **Individuals with conflicting digital reputations:** People who maintain different roles that should not mix, for example a corporate lawyer who is also a competitive streamer, or a teacher who is also active in political advocacy. These users need strict boundaries between their public-facing identities.
- **Freelancers and multi-discipline professionals:** People who work across different industries or clients and need to present relevant credentials to each audience without exposing unrelated work. A designer who also does security consulting does not want their gaming profile visible to a corporate client.
- **Privacy-conscious users:** Individuals who want to compartmentalise their online presence for safety reasons, for example keeping a legal identity separate from online communities where anonymity is expected.
- **Third-party developers:** Engineers building applications that need user profile data (like a nickname or avatar) without handling the liability of storing Personally Identifiable Information (PII). The REST API allows them to query only the relevant persona without accessing the user's full identity.

## 3.3 System Architecture

The system follows a three-tier architecture. At the top layer, two types of clients interact with the application: human users through a browser (the web interface) and external systems through HTTP requests (the REST API). Both hit the same FastAPI application server, which handles routing, authentication, input validation (via Pydantic), and the privacy enforcement logic. The application server communicates with a PostgreSQL database through SQLAlchemy ORM.



## 3.4 Design and Architectural Choices

The following sections describe the key technical decisions I made and the reasoning behind each one.

### *3.4.1 Authentication Strategy (Delegated Auth)*

**Decision:** Delegate authentication to AWS Cognito (OIDC)

**Justification:** Building a secure authentication system from scratch (handling password hashing, MFA, and session security) is error-prone and distracts from the core innovation of "Persona Management." By treating Authentication as a commodity and offloading it to AWS, the project focuses its engineering effort on the "Representation Logic."

**Current Status:** For the current iteration, I decided to implement a simpler session-based authentication using password hashing and cookies. My reasoning was that spending time on Cognito integration early on was distracting me from the core problem I wanted to solve, persona management. I wanted to get the persona logic right first, and worry about production-grade auth later. Cognito integration remains a goal for a future phase, and the design is kept abstract enough to allow swapping the auth provider (Cognito, Auth0, or Google Identity) without rewriting the persona logic.

### *3.4.2 API-First Design (REST)*

**Decision:** The core product is a RESTful API

**Justification:** Identity service is an infrastructure component. For this system to be scalable, it must be consumable by other machines (e.g., a game client requesting a user's "Gamer Profile"). A visual dashboard is provided for configuration, but the primary value is delivered via JSON responses to API consumers.

### *3.4.3 Backend Framework*

**Decision:** Python Flask.

**Justification:** Flask was selected over heavier frameworks (like Django) for its micro-framework philosophy. It allows for rapid prototyping of the routing logic and offers granular control over the HTTP response lifecycle, which is critical for implementing the custom "Context Headers" logic. Additionally, its robust extension ecosystem (Flask-SQLAlchemy, Authlib) simplifies the integration with AWS OIDC.

**Migration to FastAPI:** After working with the Flask prototype, I went ahead with the migration to FastAPI. The benefits I anticipated turned out to be real in practice:

- **Native Asynchronous Support (ASGI):** FastAPI is built on Starlette, allowing for non-blocking I/O. This matters for an identity service that may eventually need to query external APIs.

- **Strict Data Validation (Pydantic):** FastAPI uses Pydantic for request and response validation. This was a significant improvement, I defined separate response models for public responses (which exclude the `access_token`) and owner responses (which include it). Pydantic handles the filtering automatically at the framework level, which is more reliable than the manual dictionary manipulation I was doing in Flask.
- **Automatic Documentation:** FastAPI generates an interactive Swagger UI at `/docs`, which made testing the API much easier during development and would make it straightforward for third-party developers to consume.

### *3.4.4 Database Engine*

**Decision:** SQLite for the Prototype, PostgreSQL for the Current Build

**Justification:** I initially chose SQLite for prototyping because it required no setup and let me focus on the application logic. The application uses SQLAlchemy ORM, which made the eventual migration straightforward. I have since migrated to PostgreSQL running inside Docker, which gives me proper concurrent access, better JSON handling, and a setup that more closely resembles a production environment. The tests still use an in-memory SQLite database for speed and isolation.

## 3.5 Initial Database Design

This is not the final database design, this is just an initial design to build the prototype test the hypothesis.

I decided to start with two main tables representing the two main entities we have:

- **Users Table:** The biological entity/account holder
- **Personas Table:** The dimension or the projection for a certain context (e.g. A single user could have a gamer persona, a reader persona, a legal persona, and a professional persona)

### *Users Table*

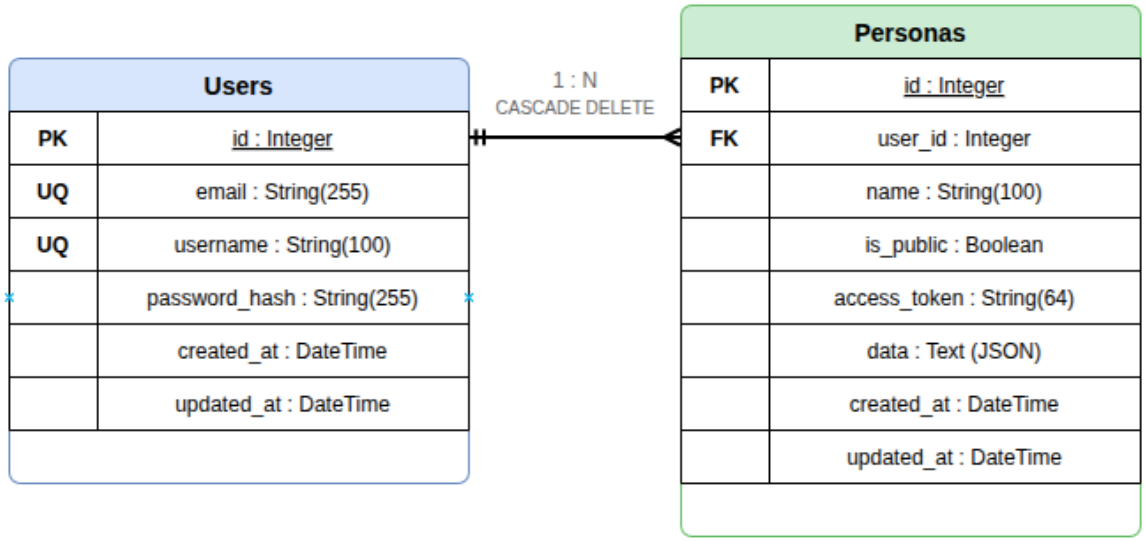
**Table 1:** Users Table Schema

| Column | Description |
|---|---|
| id | Table Primary Key |
| email | The user’s email address (unique) |
| username | A public username for the user (unique) |
| password_hash | Hashed password for session-based authentication |
| created_at | The registration date of the user |
| updated_at | Last time the user has updated their information |

### *Personas Table*

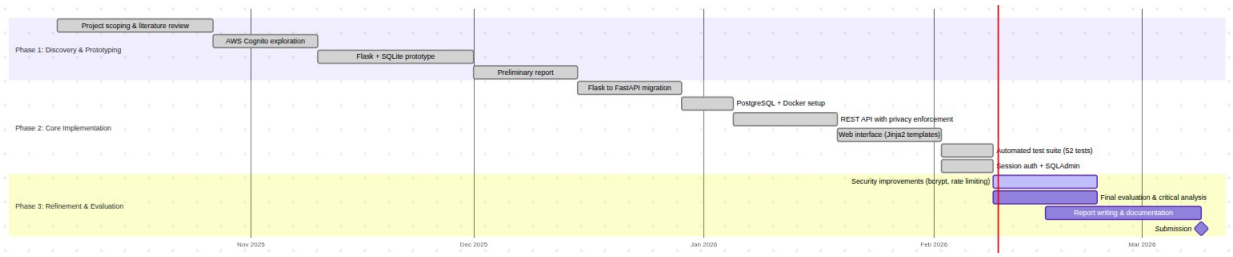
**Table 2:** Personas Table Schema

| Column | Description |
|---|---|
| id | Table Primary Key |
| user_id | A foreign key to link the persona with a user (One to many relationship) |
| name | Name of the persona (e.g. reader, gamer, legal) |
| is_public | Determines of the persona is publicly accessible for all visitors |
| access_token | Required if the persona is private, which is a security key used to access the persona |
| Data | The actual unique attributes and values of this persona, this field is JSON to keep it flexible; for example a gamer persona could have Steam ID while a professional persona could have linkedin profile or Github account |
| created_at | When was the persona created |
| updated_at | When was the last time the persona was updated |




## 3.6 Work Plan

The project execution is structured into three distinct development phases, allowing for iterative refinement and risk mitigation.




### *Phase 1: Discovery & Prototyping (Weeks 1-10) [COMPLETED]*

This initial phase focused on feasibility analysis and architectural validation. The primary objective was to test critical strategic assumptions and build a working prototype.

The key achievements of this phase were: - Having a solid vision about the project and its scope - Experimenting with AWS Cognito integration to understand the auth landscape - Building a basic prototype with Flask and SQLite, which helped me identify what worked and what needed changing

### *Phase 2: Core Implementation (Weeks 11-18) [CURRENT STATUS]*

This phase focuses on transitioning from the prototype to a proper implementation. The work done so far includes: - Migrated from Flask to FastAPI for better validation and auto-generated docs - Migrated from SQLite to PostgreSQL with Docker for a production-like setup - Implemented the full REST API with privacy enforcement (public/private personas with access tokens) - Built a web interface using Jinja2 templates for user-facing persona management - Wrote 52 automated tests using pytest - Added an admin panel using SQLAdmin - Implemented session-based authentication as an MVP (deferring Cognito to a later phase)

Remaining work for this phase: - Integrating AWS Cognito for production-grade authentication - Refactoring the monolithic `main.py` into separate route modules

### *Phase 3: Refinement & Summative Evaluation (Week 19+)*

This phase will be dedicated to assessing the system against the original design requirements and addressing remaining gaps.

Planned activities: - **Security Improvements:** Migrating from SHA256 to bcrypt for password hashing, adding rate limiting - **Product Evaluation:** Does the project solve the main problem and address the requirements? - **Documentation:** Finalizing technical documentation and the project report - **Critical Analysis:** Producing a final evaluation detailing the system's strengths, limitations, and potential for future work

## 3.7 Success Criteria & Evaluation Methodology

### *Why Automated Testing*

The core product is a REST API designed primarily for machine consumption. Unlike a user-facing GUI where subjective usability matters, an API either behaves correctly or it does not, making automated assertions the natural fit. I considered other evaluation methods: user testing would be appropriate if the web interface were the primary product, but since the API is the main deliverable, functional correctness matters more than subjective experience. Expert review could be valuable for security auditing, but at this stage the priority is verifying that the privacy model works as designed.

The test suite uses pytest and FastAPI's TestClient to simulate real HTTP requests and verify the expected responses. Tests run against an in-memory SQLite database, ensuring isolation (each test starts clean) and speed (no external dependencies).

### *Success Criteria*

Each criterion is derived from the project aims defined in section 1.2:

- **Privacy Enforcement (addresses the "Private Identity" goal):** The central promise of this project is that private personas are hidden from unauthorised access. To verify this, private personas must return `403 Forbidden` when accessed without a valid token. Tests cover three cases: no token provided, wrong token provided, and correct token provided, ensuring the privacy boundary holds in all scenarios.
- **Contextual Identity (addresses the "context collapse" problem):** The project exists because current platforms show the same data to everyone. To verify that Plural solves this, the API must return distinct data for the same user depending on the access context. Tests verify that `GET /api/users/{id}` returns only public personas, while `GET /api/personas/{id}` with the correct `X-Access-Token` header returns private persona data. This directly tests whether the system prevents context collapse.
- **Data Integrity:** Deleting a User must cascade and remove all associated Personas to prevent orphaned data. Tests create a user with multiple personas, delete the user, and verify all personas return `404 Not Found`.
- **Input Validation:** The API must reject malformed input (invalid emails, missing required fields, duplicate usernames) with appropriate error codes (`400` or `422`).

### *Scenario-Based Evaluation*

In addition to unit-level criteria, the test suite includes two scenario tests that simulate real-world use cases:

- **Recruiter Scenario:** A recruiter queries a user's profile via `GET /api/users/{id}`. The

test verifies that they see only public personas (Professional, Gamer) and have no indication that a private Legal persona exists. This directly validates the “Public multidimensional Identity” goal from section 1.2.

- **Contextual Identity Scenario:** The same user is queried by two different callers, one without a token and one with a valid access token. The test verifies that they receive different responses from the same system, proving that the API supports contextual identity projection.

#### *Web Interface Verification*

The web interface is evaluated through manual walkthrough testing, covering: user registration and login flow, creating and editing personas from the dashboard, toggling persona visibility between public and private, verifying that the public profile page (`/u/{username}`) shows only public personas, and confirming that access tokens can be regenerated from the dashboard.

---

# *4. Implementation*

## 4.1 Current Technical Stack

After the prototyping phase with Flask and SQLite, I migrated the project to a stack that better suits the requirements:

- **Web Framework:** FastAPI (running on Uvicorn, an ASGI server)
- **Database:** PostgreSQL 16 (running in Docker)
- **ORM:** SQLAlchemy with Alembic for migrations
- **Validation:** Pydantic for request/response schemas
- **Authentication:** Session-based (cookies) for the web interface, access tokens for the API
- **Templates:** Jinja2 for server-rendered HTML pages
- **Admin:** SQLAdmin for database administration
- **Containerisation:** Docker Compose to orchestrate the application and database services

## 4.2 REST API

The API is the core deliverable of the project. I implemented two groups of endpoints:

**User endpoints** handle the account lifecycle, creating, listing, reading, updating, and deleting users. When you fetch a user’s profile via `GET /api/users/{id}`, the response includes only their public personas. Private personas are filtered out at the application level before the response is sent.

**Persona endpoints** handle creating, reading, updating, and deleting personas. Each persona belongs to a user and has its own access token. Creating a persona returns the access token to the owner; after that, any mutation (update, delete, token regeneration) requires presenting the

correct token in the `X-Access-Token` header.

I also defined separate Pydantic response models: `PersonaPublicResponse` excludes the `access_token` field entirely, while `PersonaOwnerResponse` includes it. This means even if I make a mistake in the endpoint logic, Pydantic will strip the token from any public-facing response, a second layer of defence that I did not have with Flask.

## 4.3 Privacy Enforcement

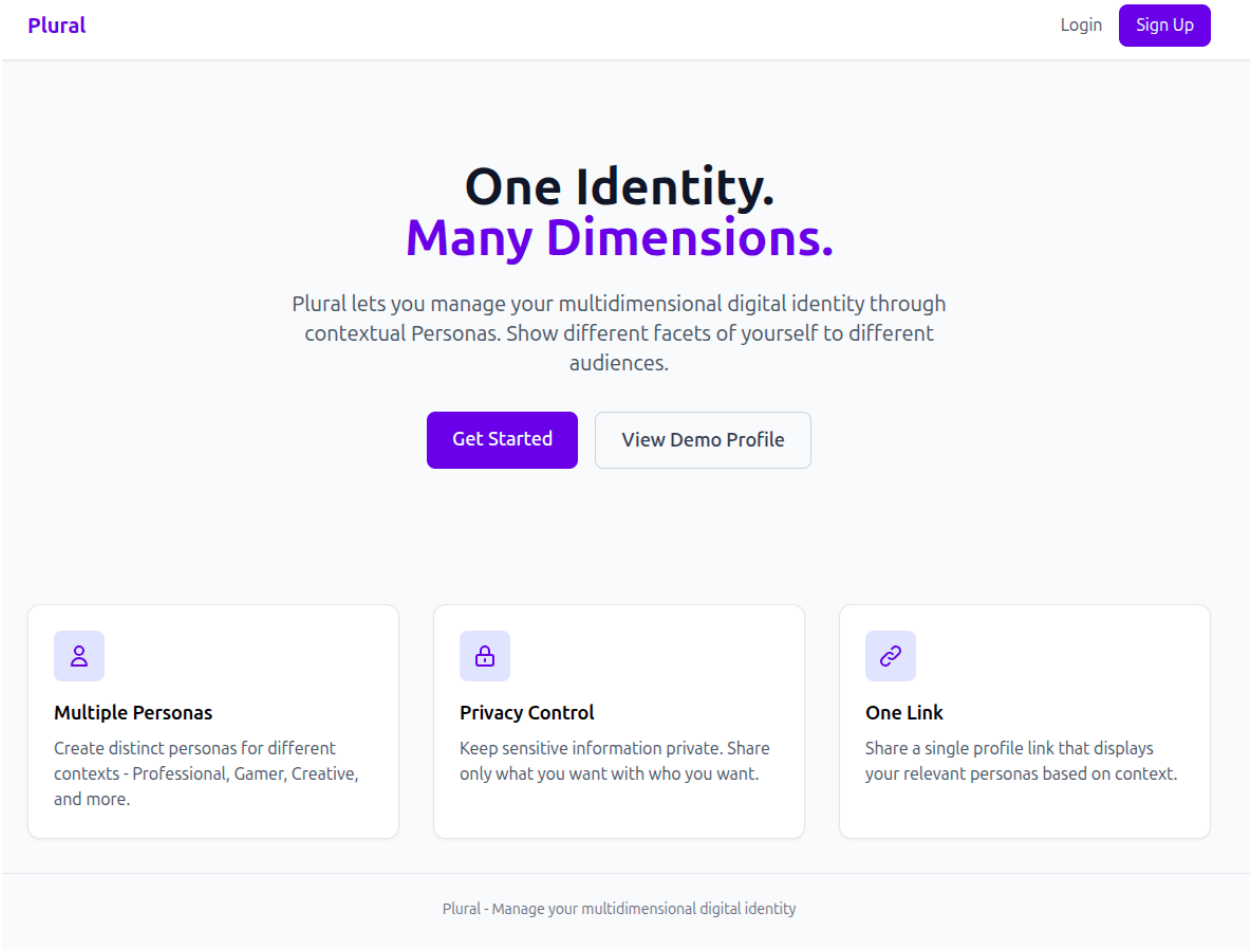
This is the core logic of the project, the part that makes it more than a basic CRUD API. The privacy model works as follows:

Each persona has two relevant fields: `is_public` (a boolean) and `access_token` (a randomly generated string). When someone requests a persona via the API:

1. If `is_public` is `True`, the persona data is returned to anyone
2. If `is_public` is `False`, the API checks for an `X-Access-Token` header
3. If no token is provided, the API returns `403 Forbidden`
4. If the token is provided but wrong, the API returns `403 Forbidden`
5. Only if the token matches does the API return the persona data

This means a single user profile can look completely different depending on the context. A recruiter browsing `GET /api/users/10` sees the Professional and Gamer personas (both public), but has no idea that a Legal persona with sensitive data even exists. Someone who was given the Legal persona's access token can retrieve it directly via `GET /api/personas/5` with the token header.

## 4.4 Web Interface

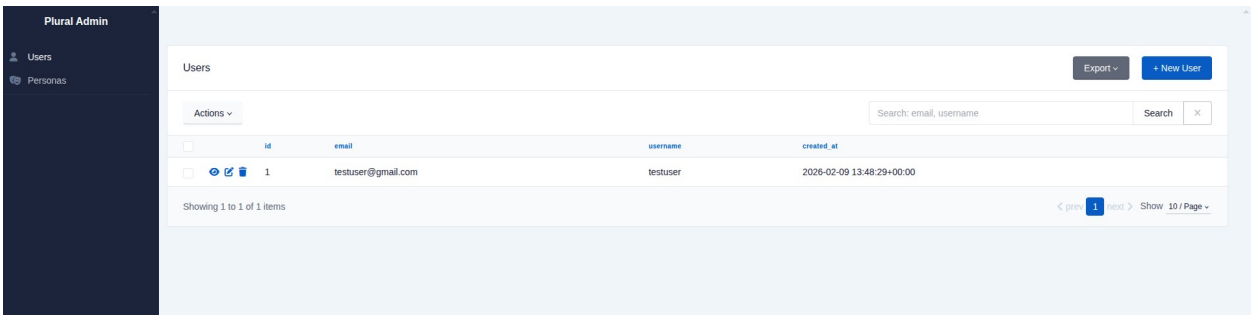



While the API is the primary product, I also built a web interface so that users can manage their personas without needing tools like Postman or curl. The interface is server-rendered using Jinja2 templates and includes:

- **Login/Signup pages:** Session-based authentication using cookies. When a user signs up, their password is hashed and stored. On login, the password is verified and a session cookie is set
- **Dashboard:** After logging in, users see all their personas (both public and private) with options to create, edit, or delete them. They can also toggle visibility and regenerate access tokens
- **Public Profile:** Each user has a shareable profile page at `/u/{username}` that shows only their public personas. This is what visitors see

All templates extend a base template (`base.html`) that provides the consistent layout and navigation.

## 4.5 Admin Panel




I integrated SQLAdmin to provide a quick way to browse and manage the database during development. It mounts at `/admin` and exposes views for both Users and Personas with search and sort functionality. This was useful for debugging and for verifying data integrity during development.

## 4.6 Flexible Data Model

One design decision I am happy with is the JSON `data` field on personas. Instead of defining fixed columns for every possible attribute (which would be impossible given the variety of persona types), I store the persona's attributes as a JSON string in a Text column. This means:

- A Gamer persona can store `{"discord": "ghosttom#4821", "rank": "plat 2"}`
- A Professional persona can store `{"linkedin": "linkedin.com/in/user", "github": "github.com/user"}`
- A Legal persona can store `{"full_name": "John Doe", "national_id": "..."}`

Each persona type has completely different fields, and the system handles them all the same way. The trade-off is that I cannot query or index individual JSON fields at the database level, but for this use case that is acceptable, the data is always accessed as a whole blob attached to a persona.

---

# *5. Evaluation*

## 5.1 Testing Approach

I wrote 52 automated tests using pytest. Rather than testing with the live PostgreSQL database, the tests run against an in-memory SQLite database. This has two advantages: the tests are fast (no network or disk I/O), and each test starts with a completely fresh database, so there is no risk of one test affecting another.

The test setup uses fixtures, reusable pieces of test state. For example, a `sample_user` fixture creates a user before the test runs, and a `sample_user_with_personas` fixture creates a user with both a public and a private persona. This keeps the individual test functions focused on

what they are actually testing.

All tests interact with the API through FastAPI's `TestClient`, which simulates HTTP requests without starting a real server. This means I am testing the full request-response cycle (routing, validation, database operations, response serialisation) without the overhead of a running server.

## 5.2 What the Tests Cover

The 52 tests are organised into the following groups, as shown in Table 3.

**Table 3:** Automated Test Suite Coverage

| Test Group | Tests | What It Covers |
|---|---|---|
| Root | 1 | Landing page loads correctly |
| User Create | 7 | Success, duplicate email/username, invalid email, missing fields, username too short |
| User List | 2 | Empty list, list with data |
| User Get | 3 | Success, not found, only public personas shown |
| User Update | 5 | Email update, username update, not found, duplicate conflicts |
| User Delete | 2 | Success, not found |
| Persona Create | 5 | Public/private creation, user not found, without data, name too short |
| Persona List | 3 | Empty, only public shown, user not found |
| Persona Get | 5 | Public without token, private without token (403), valid token, invalid token, not found |
| Persona Update | 6 | Name, visibility, data updates, missing/invalid token, not found |
| Persona Delete | 4 | Success, missing/invalid token, not found |
| Token Regeneration | 3 | Success (old token invalidated, new works), invalid token, not found |
| Cascade Delete | 1 | Deleting user removes all their personas |
| Privacy Boundaries | 2 | Recruiter scenario (only sees public), contextual identity (same user, different responses) |
| Data Integrity | 3 | Maps directly to the success criteria from section 3.7 |

## 5.3 Results Against Success Criteria

In section 3.7, I defined three concrete success criteria. Here is how the tests validate each one:

**Security Compliance ("Private Personas must return 403 Forbidden when accessed without a token"):** Multiple tests confirm this. `test_get_private_persona_without_token_forbidden` sends a GET request to a private persona without any token header and asserts a 403 response. `test_get_private_persona_with_invalid_token` does the same with a wrong token and also gets 403.

**Contextual Accuracy ("The API must return distinct attributes for the same user ID when requested with different headers"):** The `test_same_user_different_responses` test creates a user with both public and private personas, then shows that `GET /api/users/{id}` returns only the public persona, while `GET /api/personas/{id}` with the correct access token returns the private one. Same user, different views.

**Data Integrity ("Deleting a User must cascade and remove all associated Personas"):** The `test_delete_user_deletes_all_personas` test creates a user with personas, deletes the user, and verifies that both personas return 404.

## 5.4 Limitations and Improvement Plan

The project meets the success criteria I defined, but working through the implementation exposed several weaknesses. For each one, I describe the problem, why it matters, and how I would concretely fix it.

**Password Hashing (Security - High Priority)**

The current authentication uses SHA256 for password hashing. SHA256 is a general-purpose hash designed to be fast, which is the opposite of what password hashing needs, a fast hash means an attacker can try billions of guesses per second. The industry standard is bcrypt or argon2, which are deliberately slow and use salting. The fix is to replace the `hashlib.sha256` call in `auth.py` with `bcrypt.hashpw()` using `bcrypt.gensalt()`. The complication is existing users: their stored hashes cannot be reversed. The migration strategy would be to re-hash each password transparently on the next successful login, verify the old SHA256 hash, then immediately replace it with a bcrypt hash. This is a common pattern for upgrading legacy auth systems.

**No Rate Limiting (Security - High Priority)**

The API has no protection against brute-force attacks. An attacker could send thousands of login attempts per second or try to guess access tokens on the `GET /api/personas/{id}` endpoint. The fix would be to add rate limiting using a library like `slowapi` (which integrates with FastAPI) or at the reverse proxy level with nginx. The critical endpoints to protect are `/login`, `/api/personas/{id}`, and `/api/personas/{id}/regenerate-token`. A reasonable starting point would be 10 requests per minute on auth endpoints and 60 per minute on general API

endpoints.

**Authentication Layer (Architecture - Medium Priority)**

I deferred Cognito integration to focus on the persona logic, which was the right call at this stage. But the current auth lacks features users would expect: no multi-factor authentication, no social login, and no password reset flow. The integration approach would be to replace the custom login routes with an OIDC flow against Cognito's hosted UI, adding a `cognito_sub` field to the User table to link external identities to local accounts. The persona logic would remain untouched since it depends only on `user_id`, not on how the user authenticated - this separation was a deliberate design choice.

**Monolithic API File (Maintainability - Medium Priority)**

All API endpoints currently live in `main.py` (~400 lines). The web routes are already split into separate modules using FastAPI's `APIRouter`, so the pattern is established. The fix is to create `app/routes/api_users.py` and `app/routes/api_personas.py`, move the relevant endpoints, and include them via `app.include_router()`. This would reduce `main.py` to application setup only.

**No Frontend Validation (Usability - Low Priority)**

The web forms rely entirely on server-side validation, so users only see errors after a full page reload. Adding HTML5 validation attributes (`required`, `type="email"`, `minlength`) to the Jinja2 templates would catch common errors instantly. This is low priority because the REST API is the primary product and its Pydantic validation is solid.

---

# *6. Conclusion*

## 6.1 Summary

The current system allows users to create multiple personas, each with its own set of attributes and a public/private toggle. Public personas are visible to anyone who visits the user's profile. Private personas are hidden and can only be accessed with a specific access token. This means the same user profile can show completely different information depending on who is looking at it and what tokens they have.

## 6.2 What I Would Do Next

The improvements I identified in section 5.4 fall into a natural priority order. First, the security fixes (bcrypt migration and rate limiting) because they are high-impact and low-effort, both can be done without changing the application's architecture. Second, the `main.py` refactor, because the pattern is already established with the web routes and it would make the codebase easier to extend. Third, the Cognito integration, which is the largest change but is deliberately decoupled

from the persona logic.

Beyond those fixes, the feature I am most interested in is **external platform connectors**, allowing a persona to pull data from external APIs automatically. For example, a Gamer persona could sync with Steam to display current stats, or a Professional persona could pull repositories from GitHub. This would move Plural from a static data store to a live identity aggregation layer, which is closer to the original vision described in section 1.2.

## 6.3 Reflection

Looking back, the biggest lesson was learning when to defer a feature. I originally planned to integrate Cognito from the start, but that was pulling my attention away from the core problem. Switching to a simple built-in auth let me focus on getting the persona logic right, the privacy enforcement, the access tokens, the contextual responses. That is the part that makes this project different from a standard CRUD API, and it needed the most attention. The Flask-to-FastAPI migration was also worth the effort; the Pydantic validation and automatic Swagger docs saved me time and caught bugs that I would have missed with manual dictionary handling.

---

## *References*


[1] Goffman, E. (1959) *The Presentation of Self in Everyday Life*. New York: Doubleday Anchor Books.

[2] Marwick, A.E. and boyd, d. (2011) 'I Tweet Honestly, I Tweet Passionately: Twitter Users, Context Collapse, and the Imagined Audience,' *New Media & Society*, 13(1), pp. 114-133.

[3] IBM (n.d.) *What is Identity and Access Management (IAM)?* Available at: https://www.ibm.com/think/topics/identity-access-management (Accessed: 9 February 2026).

[4] SailPoint (n.d.) *Identity and Access Management.* Available at: https://www.sailpoint.com/identity-library/identity-and-access-management (Accessed: 9 February 2026).

[5] Cisco (n.d.) *What is Identity Access Management?* Available at: https://www.cisco.com/site/us/en/learn/topics/security/what-is-identity-access-management.html (Accessed: 9 February 2026).

[6] Oracle (n.d.) *What is Digital Identity?* Available at: https://www.oracle.com/middleeast/security/identity-management/digital-identity/ (Accessed: 9 February 2026).

[7] Cloudflare (n.d.) *What is Identity?* Available at: https://www.cloudflare.com/learning/access-management/what-is-identity/ (Accessed: 9 February 2026).

[8] IBM (n.d.) *What is Digital Identity?* Available at: https://www.ibm.com/think/topics/digital-identity (Accessed: 9 February 2026).

[9] Clauß, S. and Kohntopp, M. (2001) ‘Identity Management and Its Support of Multilateral Security,’ *Computer Networks*, 37, pp. 205-219.

[10] Nissenbaum, H. (2004) ‘Privacy as Contextual Integrity,’ *Washington Law Review*, 79(1), pp. 119-157.

[11] Cameron, K. (2005) *The Laws of Identity.* Microsoft Corporation. Available at: https://www.identityblog.com/stories/2005/05/13/TheLawsOfIdentity.pdf (Accessed: 9 February 2026).

[12] Alanzi, H.M. and Alkhatib, M. (2025) ‘Blockchain-Based Identity Management System Prototype for Enhanced Privacy and Security,’ *Electronics*, 14(13), 2605. doi: 10.3390/electronics14132605.

[13] Amazon Web Services (n.d.) *Federated Access.* Available at: https://docs.aws.amazon.com/whitepapers/latest/establishing-your-cloud-foundation-on-aws/federated-access.html (Accessed: 9 February 2026).

[14] Okta (n.d.) *What is Federated Identity?* Available at: https://www.okta.com/identity-101/what-is-federated-identity/ (Accessed: 9 February 2026).

[15] Entitle.io (n.d.) *What is Federated Access?* Available at: https://www.entitle.io/resources/glossary/federated-access (Accessed: 9 February 2026).